



**EDAM**

| Ethiopian Data Analytics & Modelling Consortium

---

# How to create an explainable AI pipeline

Exploring the basics of explainable AI-based models from data input to output visualisation

**Dr Gina Charnley & Dr Tristan Naidoo**

Research Associate in AI for Health Data Systems · Imperial College London · 07 September 2026



## OVERVIEW

# Agenda

---

**01** Machine and Deep Learning

---

**03** The Outcomes

---

**05** Model Fitting

---

**07** Visualisations & Outputs

---

**02** Explainable AI

---

**04** The Predictors

---

**06** Model Performance

---

**08** Wrapping Up

# 01

SECTION 01

## Machine and Deep Learning

What does it mean, how do they work, how do they differ

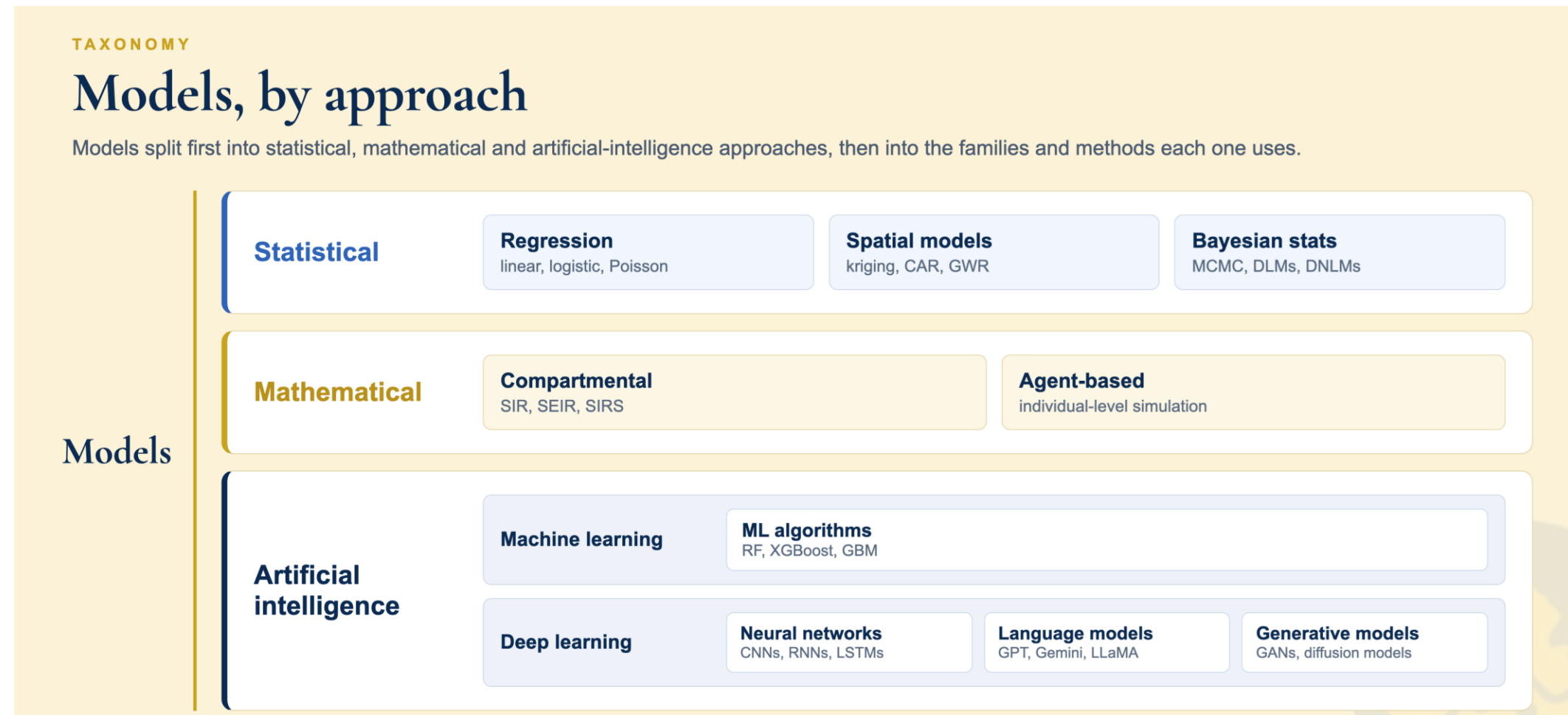


# What is a model

A model is a representation of a system – a simplification of reality

- Models are formed via a **set of assumptions**
- Encoded in **algorithms** and **equations** to capture how you believe a system works
- In epidemiology we use them to create a **computational representation of disease transmission**
- Use them to understand **risk factors**, project **future trends** and test the **effectiveness of interventions**

# Model taxonomy



- **Artificial Intelligence** sits within its **own branch of modelling**
- Distinct from mathematical and statistical modelling in **how it learns relationships**
- These approaches are **data-driven**
- Learning relationships and patterns from the data
- Rather than **inherent assumptions** (statistical distribution and mathematical parameters)



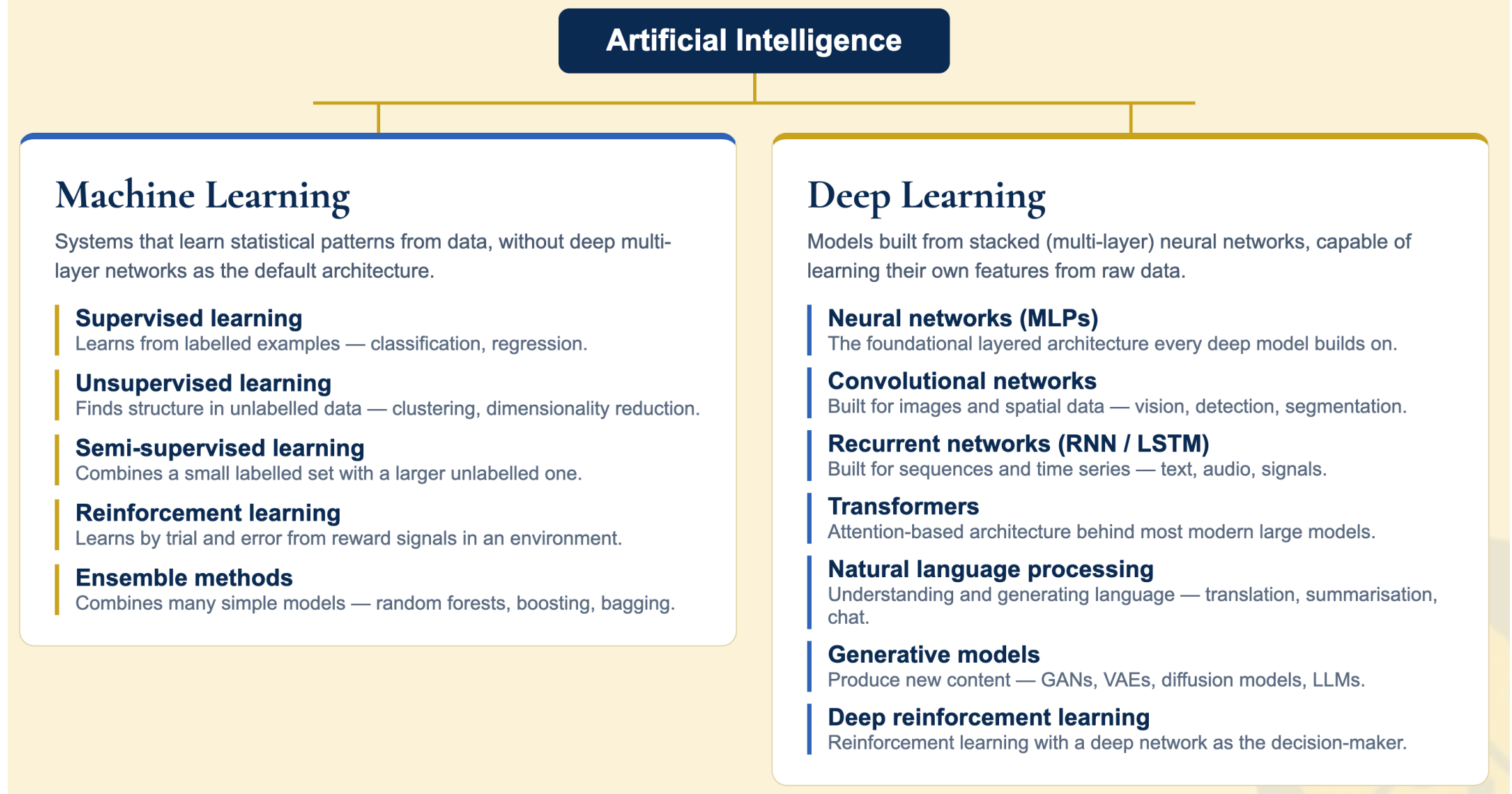
# AI-based models

- AI-based models tends to split into two main categories:
  - **Machine learning**
  - **Deep learning**
- Some consider DL a sub-category of ML
- ML is then sub-divided into:
  - **Supervised learning**
  - **Unsupervised learning**
- Supervised learning learns from **labelled answers**; unsupervised learning finds structure without any.
- Deep learning models are a **neural network** at their core

## TAXONOMY

## Breaking down AI models

The field splits first by approach — machine learning versus deep learning — then into the model families each approach produces.





SECTION 01

# Machine learning

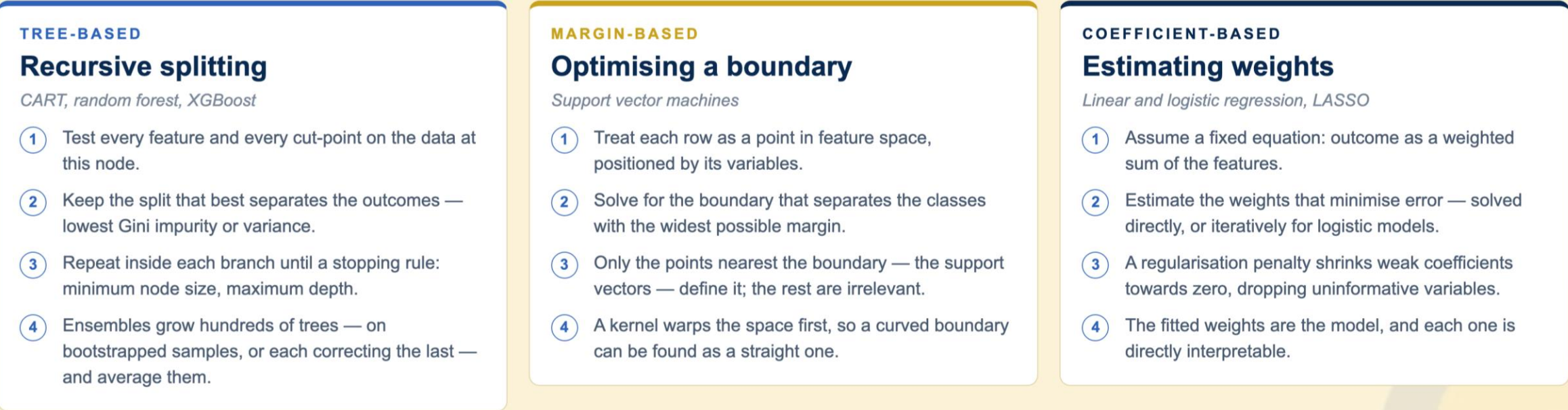
MACHINE LEARNING

## How a machine learning model is fitted

Every method takes the same three steps: read the labelled data, search for the decision rule that makes fewest mistakes on it, and freeze that rule as the model. What differs is the shape of the rule being searched for.



What the search actually does, by method family



Instance-based methods such as k-nearest neighbours skip fitting entirely — they store the training data and compare at prediction time

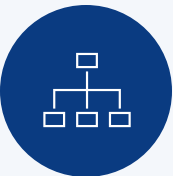


### Core idea

Every ML model follows the **same pipeline**:

1. Feed in **labelled data**
2. **Search for the rule** that fits It best
3. **Freeze that rule** as the model

Differences between algorithms comes down to **what shape of rule** they're searching for



### Rule shape

1. **Trees** — CART, RF, XGB: split the data recursively based on feature thresholds
2. **Margin-based** — SVM: treats rows as points in feature space and finds the boundary with the widest separation
3. **Coefficient-based** — LASSO: assume a fixed equation and estimate weights that minimise error

Fitting always means optimizing against training error subset to constraints that stop overfitting

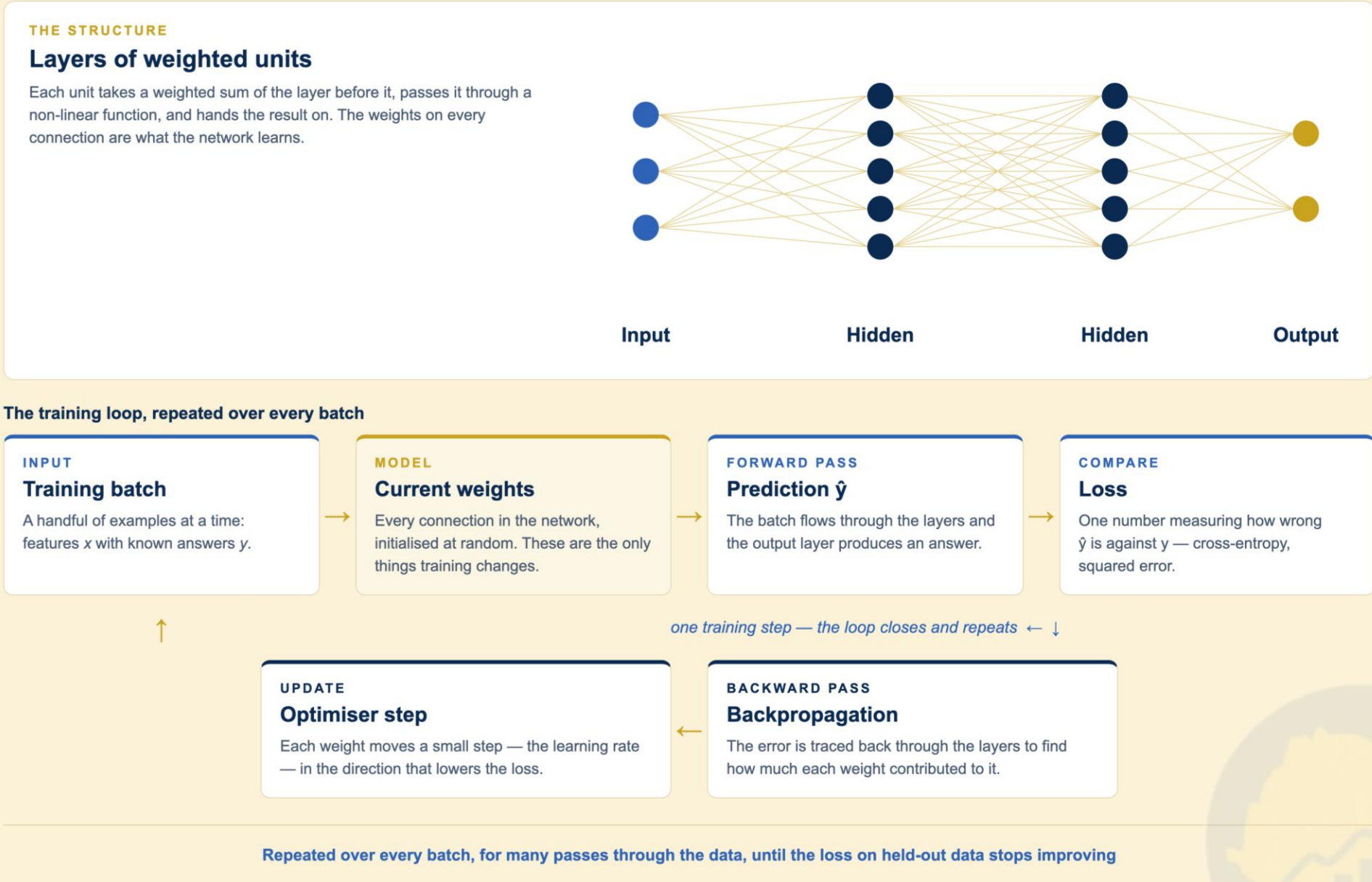
SECTION 01

# Deep learning

DEEP LEARNING

## How a neural network is trained

A neural network has no rule to search for. It starts with random weights and improves them one small step at a time, using the size of its own error to decide which direction to move.



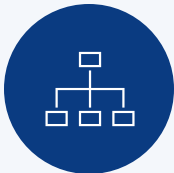
### Core idea

Deep learnings = **neural network**  
NN don't search for rules directly:

1. Starts with **random weights**
2. **Nudges** them incrementally

The **structure**:

- Layers of units
- Weighted sum of the previous layers
- Through a non-linear function
- Connection weight = learnt by the model



### Training loop

1. **Input**: takes a batch of labelled examples
2. **Forward pass**: batch with current weights are used to predict
3. **Loss**: quality how wrong they were
4. **Backward pass (backpropagation)**: trace the error back through the layers to calculate each weight's contribution
5. **Update**: Nudge every weight slightly in the direction that reduces loss

This loop repeats over batches, for many passes of the dataset (epochs), until performance on held-out data stops improving



# Main differences

## Machine learning

---

### Search strategy

- One shot search strategy

### What is being learnt

- Fit to an interpretable rule

### Feature engineering

- Features are decided in advance by you – this is the input

## Deep learning

---

- Repeated loop

- Learns weights on connections across layers

- Hidden layers effectively construct intermediate representations from the raw input

**Connecting point: training input (ground truth) + training process (minimise error)**

**Distinction: ML searches for a best-fitting rule in one shot (or a few); neural networks learn weights gradually through many small, error-corrected steps**

# When is an AI-based model the right choice

AI-based models are not always the right choice, is important to know their limitations

## When ML/DL is the **right choice**

- Prediction/classification is the goal, not explaining why
- Large datasets available - learn reliable patterns without overfitting
- Relationships are complex or non-linear

## When MD/DL is the **wrong choice**

- Causal relationships are the main question
- Small datasets - complex models are prone to overfitting
- Interpretability/regulatory justification is essential
- Underlying assumptions or mechanisms need to be tested directly

# 02

SECTION 02

## Explainable AI

What is it and how can we use it



# What is explainable AI

Explainable AI (eX-AI, XAI) is a set of methods for making model predictions interpretable

- AI-based models are criticised for being “**black-box**”
- Is a **prediction** useful **without understanding** the drivers?
- Can we trust AI model outputs without seeing the relationship it has learnt?



## Use in policy

Knowing the drivers helps with mitigation and adaptation



## Use in research

Knowing the underlying relationships can help explain poor out-of-sample performance

# XAI approaches

## Two broad approaches:

### Intrinsically interpretable models

---

Model structure is the explanation – no extra step needed

- Single decision tree – you can trace it back
- Rule-based/scoring systems

### Post-hoc explanation methods

---

Applied after fitting a complex model to explain its behaviour

- SHAP – SHapley Additive exPlanations
- LIME – Local Interpretable Model-agnostic Explanations
- PDPs – Partial Dependency Plots
- Permutation feature importance
- Counterfactual explanations



WORKED EXAMPLE

# How to create an explainable AI pipeline

Using XGBoost and SHAP values

Look out for the clock icons



## WORKED EXAMPLE

# Tree-based learning & SHAP

### MACHINE LEARNING

## How a tree-based model works

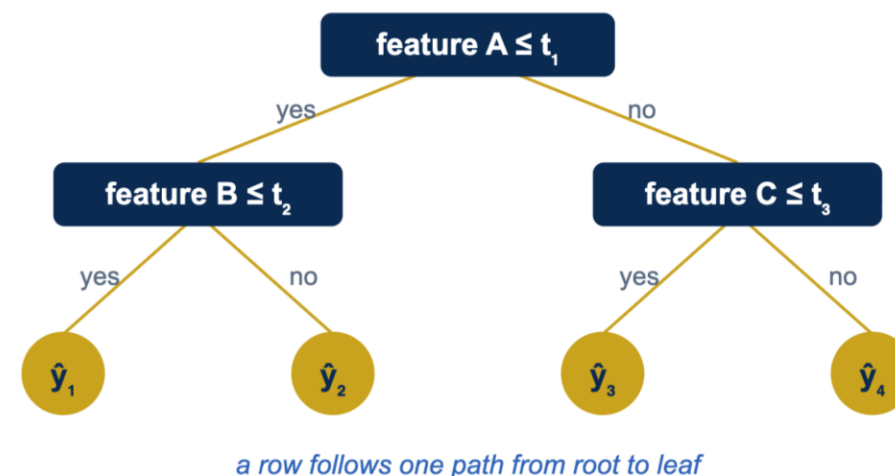
A decision tree is a sequence of yes/no questions about the features. Each question splits the data in two, and the answer at the end of a path is the model's prediction.

### THE STRUCTURE

#### Nested splits

Every internal node holds one feature and one threshold. A row is passed down the tree, taking the branch its own values satisfy, until it lands in a leaf. The leaf carries the prediction — a class, or the mean outcome of the training rows that fell there.

- Decision node — a feature and a cut-point
- Leaf — the prediction for rows reaching it



### How the tree is grown

#### STEP 1

##### Try every split

At the current node, test each feature at each possible cut-point.

#### STEP 2

##### Keep the best

Choose the split that leaves the two sides most homogeneous — lowest Gini impurity, or lowest variance.

#### STEP 3

##### Recurse

Repeat the same search independently inside each branch, on only the rows that reached it.

#### STEP 4

##### Stop and label

Halt at a depth or minimum node size, then set each leaf's prediction from its rows.

### ENSEMBLES — BAGGING

#### Random forest

Grow hundreds of trees, each on a bootstrapped sample of rows and a random subset of features, then average their predictions. Variance falls; no single tree's quirks dominate.

### ENSEMBLES — BOOSTING

#### XGBoost, LightGBM

Grow trees in sequence, each one fitted to the errors the previous trees left behind, and add them up. Usually the strongest performer on tabular data.

No gradients, no weights — the model is the set of splits, and each split is readable on its own

In the worked example we will explore a tree-based learning algorithm (XGBoost) and SHAP values

## SHAP

- Based on game theory: attributes each feature's contribution to a prediction
- Both direction and magnitude of feature's effect
- Common outputs: summary/beeswarm plots, force/waterfall plots, dependence plots
- Caveat: reflects what the model learnt, not necessarily causality

## WORKED EXAMPLE

# The workflow

1

### Data preparation

Recode binary outcome · select predictor columns · handle missingness

2

### Class imbalance check

Assess minority class proportion · consider class weights or stratified CV

3

### Feature engineering

Encode categoricals · derive composite variables · remove zero-variance

4

### Stratified train/test split

Reserve held-out test set · preserve class ratio · stratified k-fold CV

5

### Gradient boosted tree

XGBoost · native TreeSHAP support

6

### Evaluate

AUC-ROC · average precision · sensitivity · specificity · threshold tuning

7

### SHAP interpretation

Global: bar · beeswarm · Local: waterfall · Interaction: dependence plots

8

### Outputs

Ranked predictor importance · direction of effect · individual case explanations

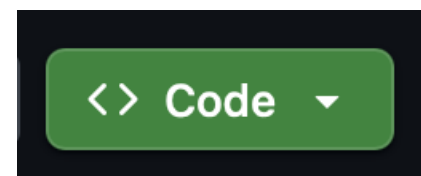
## WORKED EXAMPLE

# Downloading the repository

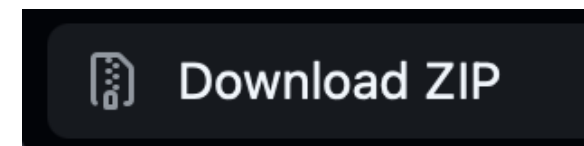
To get started with the worked example – you need to clone the repo from GitHub

1. Go to the link: <https://github.com/mrc-ide/edam-perinatal>

2. Click on the green **Code** button

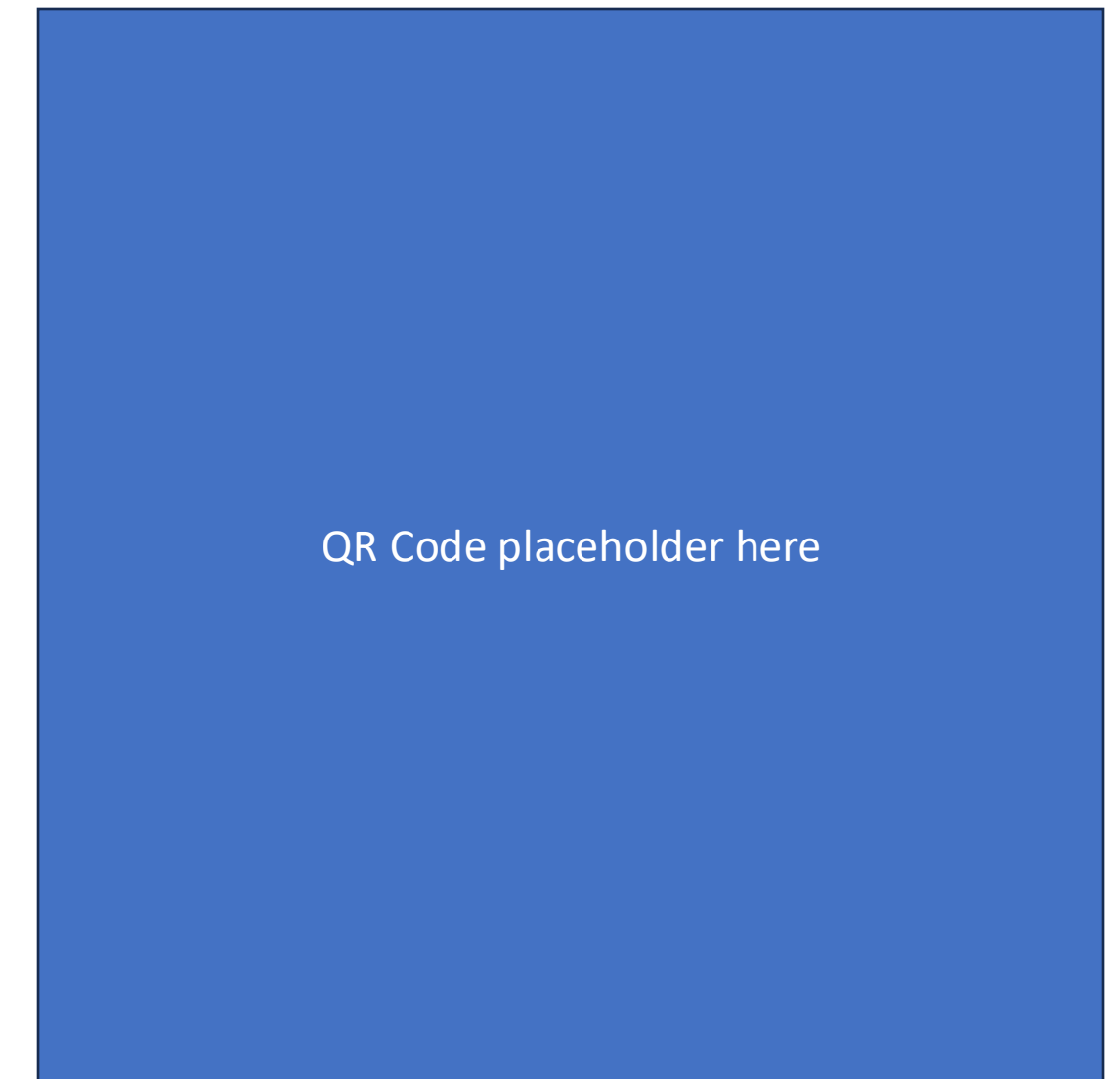


3. Select **Download ZIP**



4. Click the file, it will unzip automatically

5. Open the Rmarkdown file: **xgboost\_shap\_workflow\_tutorial.Rmd**



 **10 minutes**

# 03

SECTION 03

## The Outcomes

Defining and exploring the model outcomes





## SECTION 03

# Step 1: Data preparation

Some simple set up to help you keep track of your data and variables

In the tutorial we explore **four key steps** before using the data for fitting:

### 1. Recoding the outcome

```
recode_example %>%  
  mutate(outcome = factor(if_else(case_count > 0, "event", "no_event"),  
                           levels = c("no_event", "event")))
```

### 2. Rename coded/unintuitive columns

```
sim_data <- rename(sim_data, measurement_a = feature_1)
```

### 3. Select the columns needed for modelling

```
outcome_col <- "outcome"  
predictor_cols <- setdiff(names(sim_data), c(id_cols, outcome_col))
```

### 4. Check missingness

```
missingness <- sim_data %>%  
  select(all_of(predictor_cols)) %>%  
  summarise(across(everything(), ~ round(mean(is.na(.)) * 100, 1))) %>%  
  pivot_longer(everything(), names_to = "variable", values_to = "pct_missing") %>%  
  arrange(desc(pct_missing))
```



## SECTION 03

# Step 2: Class imbalance check

Is your outcome a rare event?

- Many health outcomes are rare – especially severe ones
- Check the minority class proportion before doing any predicting

As a rule of thumb:

- **Near-balanced outcomes** (40:60 or better): only stratified split.
- **Moderate imbalance**: use ``scale_pos_weight``.
- **Severe imbalance** (50:1 and beyond, as here): both ``scale_pos_weight`` and deliberate threshold tuning

```
n_majority <- sum(sim_data$outcome == "no_event")
n_minority <- sum(sim_data$outcome == "event")

cat("Class counts – no_event:", n_majority, " event:", n_minority, "\n")
cat("Event prevalence:", round(n_minority / (n_majority + n_minority) * 100, 2), "%\n")
cat("Imbalance ratio (majority:minority):", round(n_majority / n_minority, 1), ":1\n")

# sqrt of the ratio is a common, less aggressive alternative to the raw
# ratio for XGBoost's scale_pos_weight – it upweights the minority class
# without letting it dominate the loss entirely
weight_ratio <- sqrt(n_majority / n_minority)
cat("scale_pos_weight (sqrt of ratio):", round(weight_ratio, 2), "\n")
...
```

```
Class counts – no_event: 1940  event: 60
Event prevalence: 3 %
Imbalance ratio (majority:minority): 32.3 :1
scale_pos_weight (sqrt of ratio): 5.69
```

 **15 minutes**

## WORKED EXAMPLE

# The workflow

1



### Data preparation

Recode binary outcome · select predictor columns · handle missingness

2



### Class imbalance check

Assess minority class proportion · consider class weights or stratified CV

3

### Feature engineering

Encode categoricals · derive composite variables · remove zero-variance

4

### Stratified train/test split

Reserve held-out test set · preserve class ratio · stratified k-fold CV

5

### Gradient boosted tree

XGBoost · native TreeSHAP support

6

### Evaluate

AUC-ROC · average precision · sensitivity · specificity · threshold tuning

7

### SHAP interpretation

Global: bar · beeswarm · Local: waterfall · Interaction: dependence plots

8

### Outputs

Ranked predictor importance · direction of effect · individual case explanations

# 04

SECTION 04

## The Predictors

Exploring multi-collinearity and defining the testing and training data



## SECTION 04

# Step 3: Feature engineering

Similar to the outcomes, the features also need some work

In the tutorial we explore **four key steps** for feature engineering:

1. Deriving composite variables

```
mutate(  
  flag_any = if_else(  
    rowSums(!is.na(across(c(flag_x1, flag_x2, flag_x3)))) == 0,  
    NA_real_,  
    if_else(rowSums(across(c(flag_x1, flag_x2, flag_x3), ~ . == 1),  
      na.rm = TRUE) > 0, 1, 0)
```

2. Assessing multicollinearity

```
cor_matrix <- sim_data %>%  
  select(all_of(numeric_predictors)) %>%  
  cor(use = "pairwise.complete.obs")
```

3. Identify and drop volume markers

```
vars_to_drop <- c(volume_markers, zero_variance)  
predictor_cols <- predictor_cols[!predictor_cols %in% vars_to_drop]
```

4. Build the modelling frame

```
model_data <- sim_data %>%  
  select(all_of(id_cols), all_of(outcome_col), all_of(predictor_cols))
```



**15 minutes**

## WORKED EXAMPLE

# The workflow

1



### Data preparation

Recode binary outcome · select predictor columns · handle missingness

2



### Class imbalance check

Assess minority class proportion · consider class weights or stratified CV

3



### Feature engineering

Encode categoricals · derive composite variables · remove zero-variance

4

### Stratified train/test split

Reserve held-out test set · preserve class ratio · stratified k-fold CV

5

### Gradient boosted tree

XGBoost · native TreeSHAP support

6

### Evaluate

AUC-ROC · average precision · sensitivity · specificity · threshold tuning

7

### SHAP interpretation

Global: bar · beeswarm · Local: waterfall · Interaction: dependence plots

8

### Outputs

Ranked predictor importance · direction of effect · individual case explanations



# 05

SECTION 05

## Model Fitting

Model fitting and SHAP value extraction



## Step 4: Stratified train/test split

Machine learning models needs a training dataset and a testing dataset

- Common training/testing splits are about **70-80%**
- Allowing you to test your model on a decent amount of unseen data
- You should **stratify this split**: ensure number of outcomes are representative
- Especially when your outcome is a rare event
- Important to also consider this in other splits: here we are using cross-validation folds

```
set.seed(42)
split      <- initial_split(model_data, prop = 0.8, strata = outcome)
train_data <- training(split)
test_data  <- testing(split)

cv_folds <- vfold_cv(train_data, v = 10, strata = outcome)

cat("Train:", nrow(train_data), " Test:", nrow(test_data), "\n")
cat("Train event rate:", round(mean(train_data$outcome == "event") * 100, 2), "%\n")
cat("Test event rate:",  round(mean(test_data$outcome  == "event") * 100, 2), "%\n")
...
```

```
Train: 1600 Test: 400
Train event rate: 2.88 %
Test event rate: 3.5 %
```



## SECTION 05

# Step 5: Gradient boosted tree

Here, we are basically creating the model recipe

```
xgb_recipe <- recipe(outcome ~ ., data = train_data) %>%  
  update_role(all_of(id_cols), new_role = "ID") %>%  
  step_nominal(all_nominal_predictors()) %>%  
  step_unknown(all_nominal_predictors()) %>%  
  step_dummy(all_nominal_predictors()) %>%  
  step_impute_median(measurement_a, feature_8) %>%  
  step_zv(all_predictors())  
  
xgb_spec <- boost_tree(  
  trees      = 500,  
  tree_depth = 4,  
  learn_rate = 0.05,  
  loss_reduction = 0.01,  
  min_n      = 10  
) %>%  
  set_engine("xgboost", scale_pos_weight = weight_ratio) %>%  
  set_mode("classification")  
  
xgb_workflow <- workflow() %>%  
  add_recipe(xgb_recipe) %>%  
  add_model(xgb_spec)
```

Note the parameters

The recipe:

1. Encodes categoricals
2. Handles unseen factor levels  
Handles missingness
3. Imputes the numeric columns that are missing-at-random
4. Drops any residual zero-variance columns as a safety net.
5. Identifier columns prevented from being carried through as predictors.

 **20 minutes**

## WORKED EXAMPLE

# The workflow

1



### Data preparation

Recode binary outcome · select predictor columns · handle missingness

2



### Class imbalance check

Assess minority class proportion · consider class weights or stratified CV

3



### Feature engineering

Encode categoricals · derive composite variables · remove zero-variance

4



### Stratified train/test split

Reserve held-out test set · preserve class ratio · stratified k-fold CV

5



### Gradient boosted tree

XGBoost · native TreeSHAP support

6

### Evaluate

AUC-ROC · average precision · sensitivity · specificity · threshold tuning

7

### SHAP interpretation

Global: bar · beeswarm · Local: waterfall · Interaction: dependence plots

8

### Outputs

Ranked predictor importance · direction of effect · individual case explanations

# 06

SECTION 06

## Model Performance

Extracting the model performance metrics and model tuning



## Step 6: Evaluate

We can assess how models perform in multiple ways

Here, we explore model performance in two ways:

### 1. Cross-validation performance

1. The training set is split into k-folds (chunks of data)
2. You refit and retest your model across these folds

### 2. Threshold tuning

1. The model default assume the probability of an event is 50%
2. In rare events, this is not the case
3. You test various threshold options

Note: there are other parameters that can be important to tune

Then you fit your best model and extract the performance metrics on the **testing dataset**



SECTION 06

Performance metrics can be subject to interpretation

Defining “good enough” can be tricky

Metrics explained (with our numbers):

- **ROC AUC:** Overall ability to discriminate between the two outcomes; 0.5 = random guessing, 1.0 = perfect. **Very weak** 0.551/1
- **Sensitivity:** Of all actual events, the proportion the model correctly captured. **Very poor** 0.0714/1
- **Specificity:** Of all actual non-events, the proportion correctly identified. **Good** 0.951/1
- **PPV:** Of all cases the model predicted as events, the proportion that were actually events. **Very poor** 0.0500/1
- **NPV:** Of all cases predicted as "no event," the proportion that really were no event. **Good** 0.966/1
- **F-measure:** Harmonic mean of precision and sensitivity. **Very poor** 0.0588/1

Note that NPV and specificity are easy in rare events.  
Performance metrics depend on your outcome and model

|                           |             |                      |
|---------------------------|-------------|----------------------|
| # A tibble: 6 × 3         |             |                      |
|                           | .metric     | .estimator .estimate |
|                           | <chr>       | <chr> <dbl>          |
| 1                         | roc_auc     | binary 0.551         |
| 2                         | sensitivity | binary 0.0714        |
| 3                         | specificity | binary 0.951         |
| 4                         | ppv         | binary 0.0500        |
| 5                         | npv         | binary 0.966         |
| 6                         | f_meas      | binary 0.0588        |
| Truth                     |             |                      |
| Prediction no_event event |             |                      |
|                           | no_event    | 367 13               |
|                           | event       | 19 1                 |



## WORKED EXAMPLE

# The workflow

1



### Data preparation

Recode binary outcome · select predictor columns · handle missingness

2



### Class imbalance check

Assess minority class proportion · consider class weights or stratified CV

3



### Feature engineering

Encode categoricals · derive composite variables · remove zero-variance

4



### Stratified train/test split

Reserve held-out test set · preserve class ratio · stratified k-fold CV

5



### Gradient boosted tree

XGBoost · native TreeSHAP support

6



### Evaluate

AUC-ROC · average precision · sensitivity · specificity · threshold tuning

7

### SHAP interpretation

Global: bar · beeswarm · Local: waterfall · Interaction: dependence plots

8

### Outputs

Ranked predictor importance · direction of effect · individual case explanations

# 07

SECTION 07

## Visualisations & Outputs

Visualising the model outputs





## Step 7: SHAP interpretation

There are multiple ways to explore your SHAP values

XGBoost has built in SHAP capabilities

```
shp <- shapviz(xgb_model, X_pred = train_baked)
```

In the tutorial, we explore them in four plots:

**1. Bar plot** – Global importance

- Ranks predictors by their overall impact on model predictions

**2. Beeswarm plot** – Global importance + Direction

- Same ranking as the bar plot, but each dot is an individual case, showing both the magnitude and direction

**3. Dependence plots** – Marginal effects

- Show how the model's prediction changes across the range of a single predictor

**4. Waterfall plot** – Local explanations for a single feature

- Explains one individual prediction rather than the model overall

## SECTION 07

# Step 8: Outputs

**Always good to have the raw outputs stored, not just the plots**

The end product of Steps 1–7 is a set of reusable outputs:

- A ranked predictor importance table
- The direction of each predictor's effect
- Individual case explanations

Typically exported for reporting rather than kept only as plot objects.

These can be reported in your results and discussion as exact values along with your performance metrics from Step 6.



## WORKED EXAMPLE

# The workflow

1

### Data preparation

Recode binary outcome · select predictor columns · handle missingness



2

### Class imbalance check

Assess minority class proportion · consider class weights or stratified CV



3

### Feature engineering

Encode categoricals · derive composite variables · remove zero-variance



4

### Stratified train/test split

Reserve held-out test set · preserve class ratio · stratified k-fold CV



5

### Gradient boosted tree

XGBoost · native TreeSHAP support



6

### Evaluate

AUC-ROC · average precision · sensitivity · specificity · threshold tuning



7

### SHAP interpretation

Global: bar · beeswarm · Local: waterfall · Interaction: dependence plots



8

### Outputs

Ranked predictor importance · direction of effect · individual case explanations



# 08

SECTION 08

## Wrapping Up

Bringing it all together and key takeaways



## SECTION 08

# Key takeaways

## Bringing it all together, key take home points



### Know your data

Explore it before fitting so your model performs at its best.



### Pick the right model

Match the model to the question, not just the data.



### Keep it reproducible

Keep notes as you go so your results can be repeated.



### Mind class balance

It drives your splitting, weighting and threshold choices.



### Look beyond one metric

No single metric tells the whole story — report them together.



### Explainability ≠ causation

SHAP shows what the model learned, not the true cause.



## SECTION 08

# Taking it further

This is to get started, but there are more ways to improve the robustness of your models



### Respect spatial & temporal structure

Standard CV can leak across space or time — use block or time-aware splits (Moran's I).



### Tune hyperparameters

Go beyond defaults with grid, random or Bayesian search (e.g. the tune package).



### Explore ensemble methods

Combine models for robustness with packages like caretEnsemble.



### Watch for multicollinearity

Other ML methods can be more sensitive to correlated predictors.



### Cross-check explanations

Compare SHAP against LIME or permutation importance to test robustness.



### Validate thoroughly

Check calibration, not just discrimination, and use repeated or nested CV.



**EDAM**

Ethiopian Data Analytics & Modelling Consortium

# Thank you

[info@edamconsortium.org](mailto:info@edamconsortium.org)

[www.edamconsortium.org](http://www.edamconsortium.org)

